

CSA0617-Design&Analysis ofAlgorithms

Day 5-Lab Programming DAA

1. 0/1 Knapsack Problem – Dynamic Programming

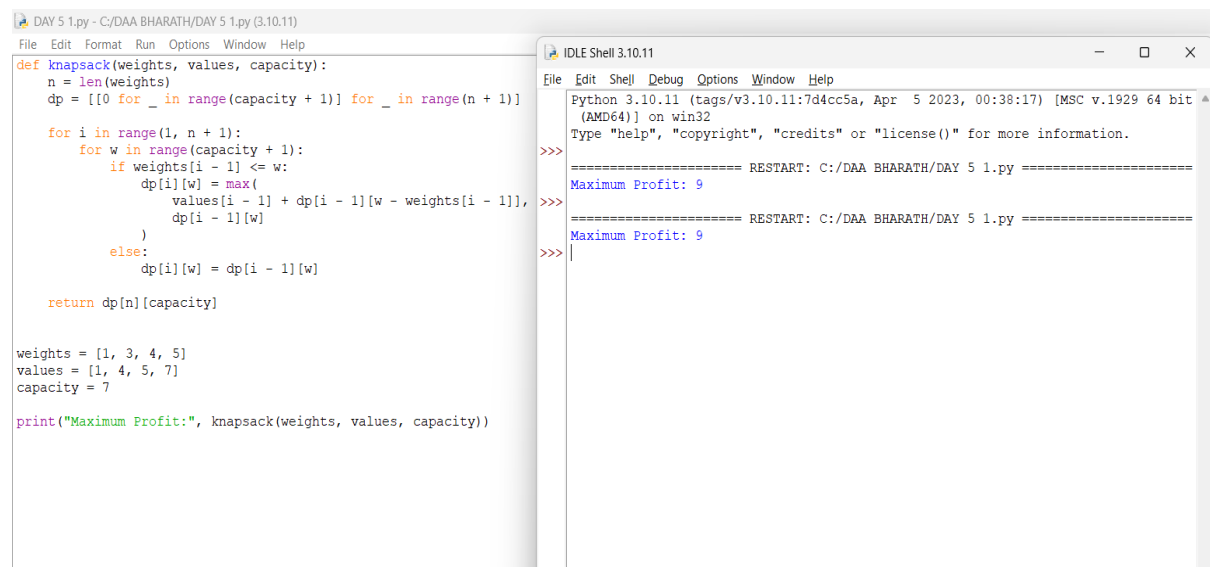
Aim

To implement the **0/1 Knapsack Problem using Dynamic Programming** and find the maximum profit that can be obtained within the given capacity.

Algorithm

1. Start with the weights, values, and knapsack capacity.
2. Create a DP table $dp[i][w]$.
3. Initialize all values to 0.
4. For each item, check whether its weight is less than or equal to the current capacity.
5. If possible, choose the maximum of including or excluding the item.
6. Otherwise, exclude the item.
7. Return $dp[n][capacity]$.

Program & Output



```
DAY 5 1.py - C:/DAA BHARATH/DAY 5 1.py (3.10.11)
File Edit Format Run Options Window Help
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7

print("Maximum Profit:", knapsack(weights, values, capacity))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 5 1.py =====
Maximum Profit: 9
>>>
===== RESTART: C:/DAA BHARATH/DAY 5 1.py =====
Maximum Profit: 9
>>>
```

2. Travelling Salesperson Problem – Dynamic Programming

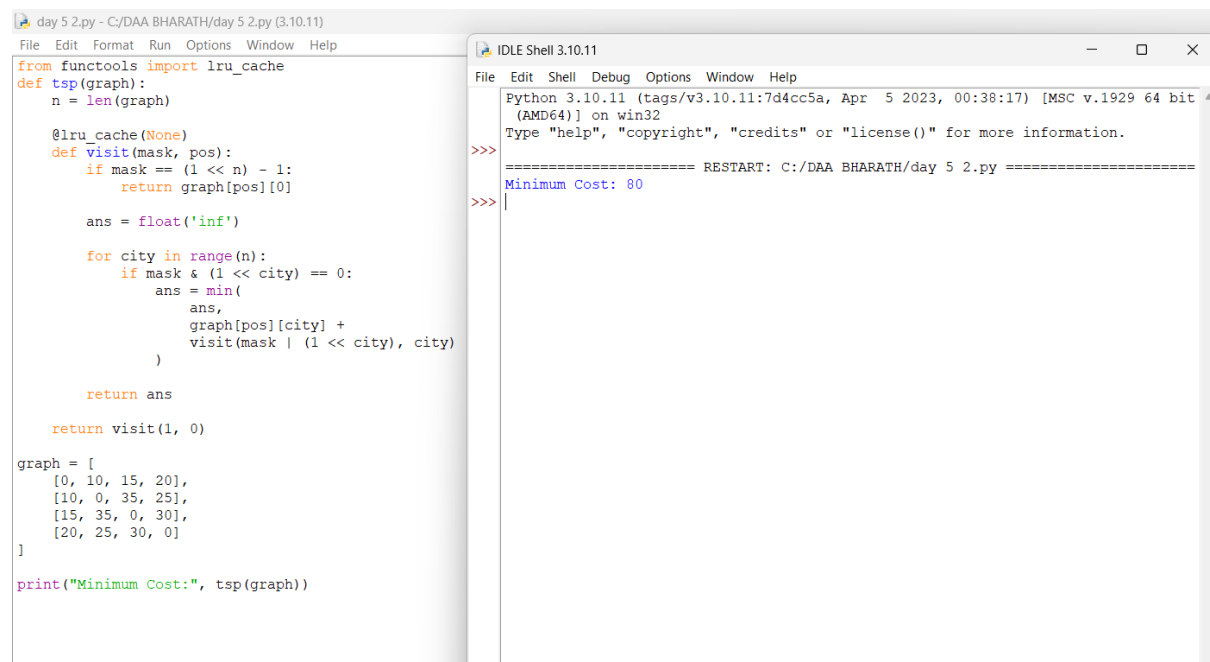
Aim

To implement the Travelling Salesperson Problem using Dynamic Programming and find the minimum cost tour visiting all cities exactly once.

Algorithm

1. Represent the cities and distances using a cost matrix.
2. Use a bit mask to represent visited cities.
3. Start from city 0.
4. Recursively visit every unvisited city.
5. Calculate the minimum cost for each possible route.
6. Return to the starting city after visiting all cities.
7. Select the minimum total cost.

Program & Output



The screenshot shows a Python IDE with two windows. The left window displays the code for the Travelling Salesperson Problem, and the right window shows the output.

```
day 5 2.py - C:/DAA BHARATH/day 5 2.py (3.10.11)
File Edit Format Run Options Window Help
from functools import lru_cache
def tsp(graph):
    n = len(graph)

    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]

        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(
                    ans,
                    graph[pos][city] +
                    visit(mask | (1 << city), city)
                )

        return ans

    return visit(1, 0)

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

print("Minimum Cost:", tsp(graph))
```

The right window shows the output of the program:

```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/day 5 2.py =====
Minimum Cost: 80
>>>
```

3. Optimal Binary Search Tree – OBST

Aim

To construct an Optimal Binary Search Tree using Dynamic Programming and find the minimum search cost.

Algorithm

1. Store the frequency of each key.

2. Create a cost table.
3. Initialize the cost of a single key with its frequency.
4. Consider chains of increasing length.
5. Try every key as the root.
6. Calculate left subtree cost, right subtree cost, and total frequency.
7. Select the minimum cost.
8. Return the cost of the complete tree.

Program & Output

```

day 5 3.py - C:/DAA BHARATH/day 5 3.py (3.10.11)
File Edit Format Run Options Window Help
def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')

            total_freq = sum(freq[i:j + 1])

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0

                cost[i][j] = min(
                    cost[i][j],
                    left + right + total_freq
                )

    return cost[0][n - 1]

keys = [10, 20, 30]
freq = [34, 8, 50]

print("Optimal BST Cost:", optimal_bst(keys, freq))

```

```

IDLE Shell 3.10.11
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/day 5 3.py =====
Optimal BST Cost: 142
>>>
===== RESTART: C:/DAA BHARATH/day 5 3.py =====
Optimal BST Cost: 142
>>>

```

4. 0/1 Knapsack Problem

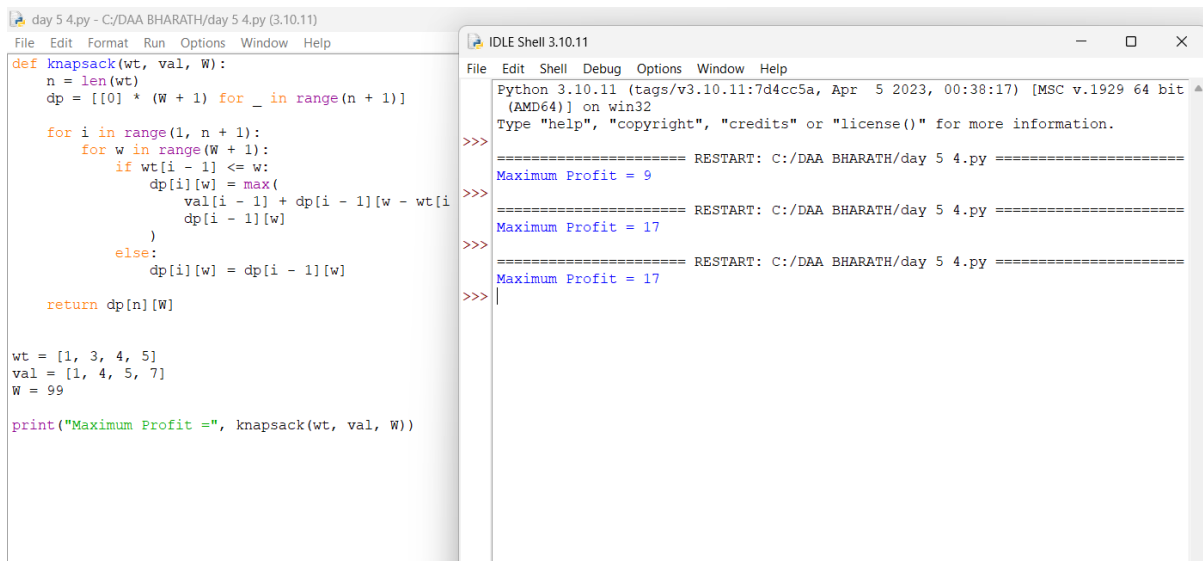
Aim

To implement the 0/1 Knapsack Problem using Dynamic Programming and determine the maximum possible profit.

Algorithm

1. Read weights, values, and capacity.
2. Create a DP table.
3. Consider each item one by one.
4. If the item fits, calculate both inclusion and exclusion cases.
5. Store the maximum value.
6. If the item does not fit, copy the previous value.
7. Return the final maximum profit.

Program & Output



```
day 5 4.py - C:/DAA BHARATH/day 5 4.py (3.10.11)
File Edit Format Run Options Window Help

def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if wt[i - 1] <= w:
                dp[i][w] = max(
                    val[i - 1] + dp[i - 1][w - wt[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 99

print("Maximum Profit =", knapsack(wt, val, W))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/DAA BHARATH/day 5 4.py =====
Maximum Profit = 9
>>>
===== RESTART: C:/DAA BHARATH/day 5 4.py =====
Maximum Profit = 17
>>>
===== RESTART: C:/DAA BHARATH/day 5 4.py =====
Maximum Profit = 17
>>>
```

5. 0/1 Knapsack Using Memoization

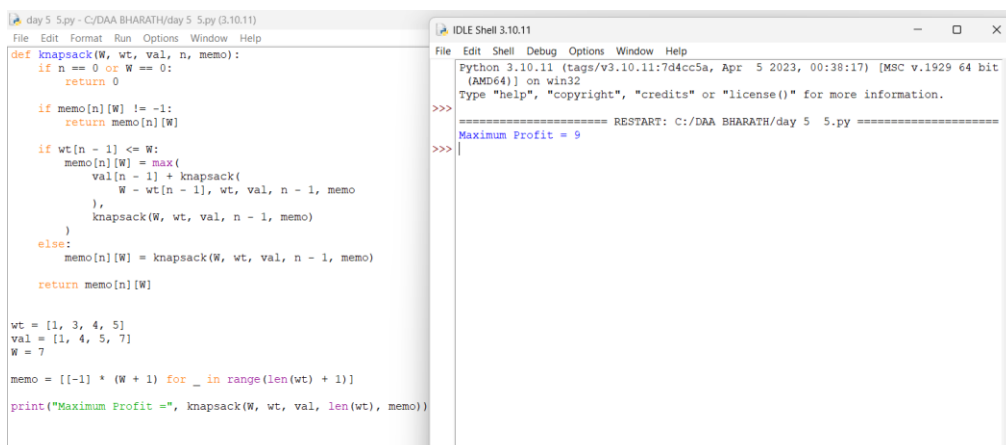
Aim

To implement the 0/1 Knapsack Problem using Memoization and find the maximum achievable profit.

Algorithm

1. Initialize a memoization table with -1.
2. Check whether capacity or number of items is zero.
3. If the result is already stored, return it. If the item fits, consider including and excluding it.
4. Store the maximum result in the memo table. If the item does not fit, exclude it.
5. Return the calculated maximum profit.

Program & Output



```
day 5 5.py - C:/DAA BHARATH/day 5 5.py (3.10.11)
File Edit Format Run Options Window Help

def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0

    if memo[n][W] != -1:
        return memo[n][W]

    if wt[n - 1] <= W:
        memo[n][W] = max(
            val[n - 1] + knapsack(
                W - wt[n - 1], wt, val, n - 1, memo
            ),
            knapsack(W, wt, val, n - 1, memo)
        )
    else:
        memo[n][W] = knapsack(W, wt, val, n - 1, memo)

    return memo[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7

memo = [[-1] * (W + 1) for _ in range(len(wt) + 1)]

print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/DAA BHARATH/day 5 5.py =====
Maximum Profit = 9
>>>
```

6. Subset Sum Problem

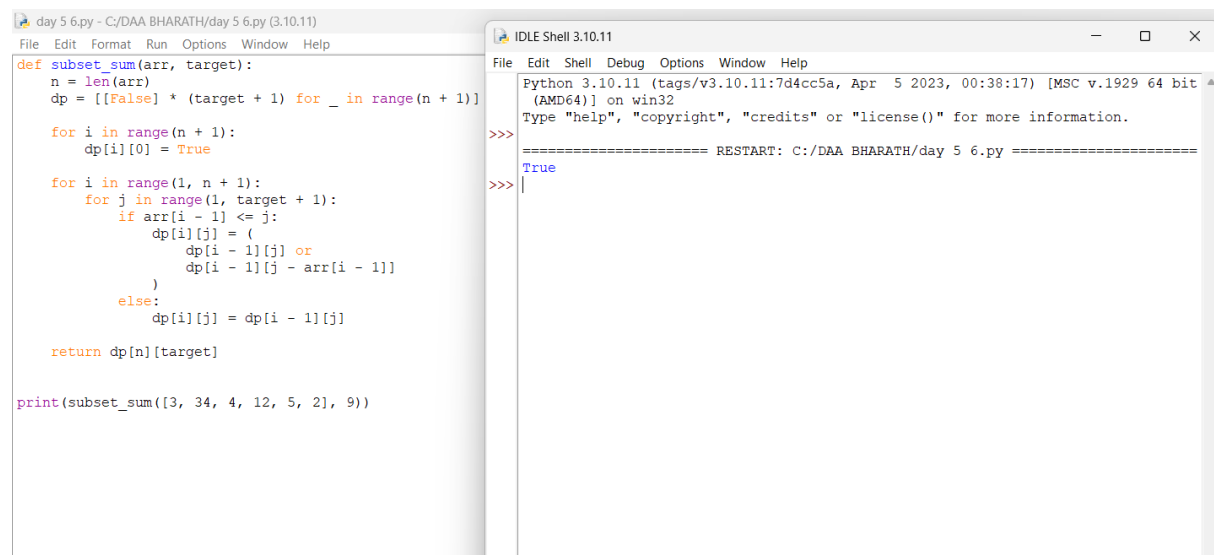
Aim

To implement the Subset Sum Problem using Dynamic Programming and determine whether a subset with the given target sum exists.

Algorithm

1. Read the array and target sum.
2. Create a Boolean DP table.
3. Set the first column as True.
4. For each element, check whether it can be included.
5. Store True if the target can be formed.
6. Continue until all elements are processed.
7. Return the final Boolean result.

Program & Output



The image shows a screenshot of a Python IDE with two windows. The left window, titled 'day 5 6.py - C:/DAA BHARATH/day 5 6.py (3.10.11)', contains the following Python code:

```
def subset_sum(arr, target):
    n = len(arr)
    dp = [[False] * (target + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for j in range(1, target + 1):
            if arr[i - 1] <= j:
                dp[i][j] = (
                    dp[i - 1][j] or
                    dp[i - 1][j - arr[i - 1]]
                )
            else:
                dp[i][j] = dp[i - 1][j]

    return dp[n][target]

print(subset_sum([3, 34, 4, 12, 5, 2], 9))
```

The right window, titled 'IDLE Shell 3.10.11', shows the output of the program:

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/day 5 6.py =====
True
>>>
```

7. Travelling Salesperson Problem – DP Version

Aim

To implement the Travelling Salesperson Problem using Dynamic Programming with Bit Masking and find the minimum tour cost.

Algorithm

1. Store the distance matrix.
2. Represent visited cities using a bit mask.

3. Start from city 0.
4. Select each unvisited city.
5. Recursively calculate the minimum route.
6. Return to city 0 after visiting all cities.
7. Print the minimum tour cost.

Program & Output

The screenshot shows a Python IDE with two windows. The left window, titled 'day 5 7.py', contains the following code:

```
from functools import lru_cache

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)

@lru_cache(None)
def tsp(mask, pos):
    if mask == (1 << n) - 1:
        return graph[pos][0]

    ans = float('inf')

    for city in range(n):
        if not mask & (1 << city):
            ans = min(
                ans,
                graph[pos][city] +
                tsp(mask | (1 << city), city)
            )

    return ans

print("Minimum Cost =", tsp(1, 0))
```

The right window, titled 'IDLE Shell 3.10.11', shows the output of the program:

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/day 5 7.py =====
Minimum Cost = 80
>>>
```

8. Travelling Salesperson Using DP Table

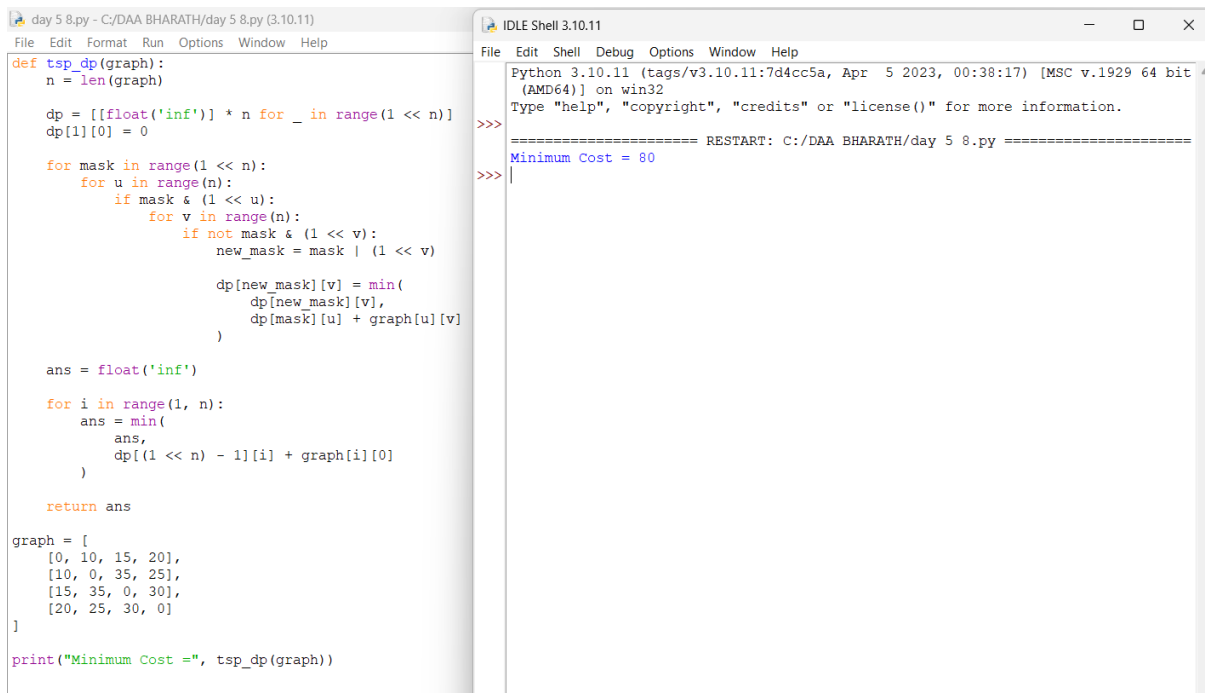
Aim

To implement the Travelling Salesperson Problem using a Dynamic Programming Table and determine the minimum travelling cost.

Algorithm

1. Create a DP table using subsets of cities.
2. Initialize the starting city with cost 0.
3. For every visited-city set, consider all current cities.
4. Try every unvisited city as the next city.
5. Update the DP table with the minimum cost.
6. After visiting all cities, add the cost of returning to city 0.
7. Select the minimum total cost.

Program & Output



```
day 5 8.py - C:/DAA BHARATH/day 5 8.py (3.10.11)
File Edit Format Run Options Window Help

def tsp_dp(graph):
    n = len(graph)

    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0

    for mask in range(1 << n):
        for u in range(n):
            if mask & (1 << u):
                for v in range(n):
                    if not mask & (1 << v):
                        new_mask = mask | (1 << v)

                        dp[new_mask][v] = min(
                            dp[new_mask][v],
                            dp[mask][u] + graph[u][v]
                        )

    ans = float('inf')

    for i in range(1, n):
        ans = min(
            ans,
            dp[(1 << n) - 1][i] + graph[i][0]
        )

    return ans

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

print("Minimum Cost =", tsp_dp(graph))

IDLE Shell 3.10.11
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/day 5 8.py =====
>>>
Minimum Cost = 80
>>>
```

9. Optimal Binary Search Tree – OBST

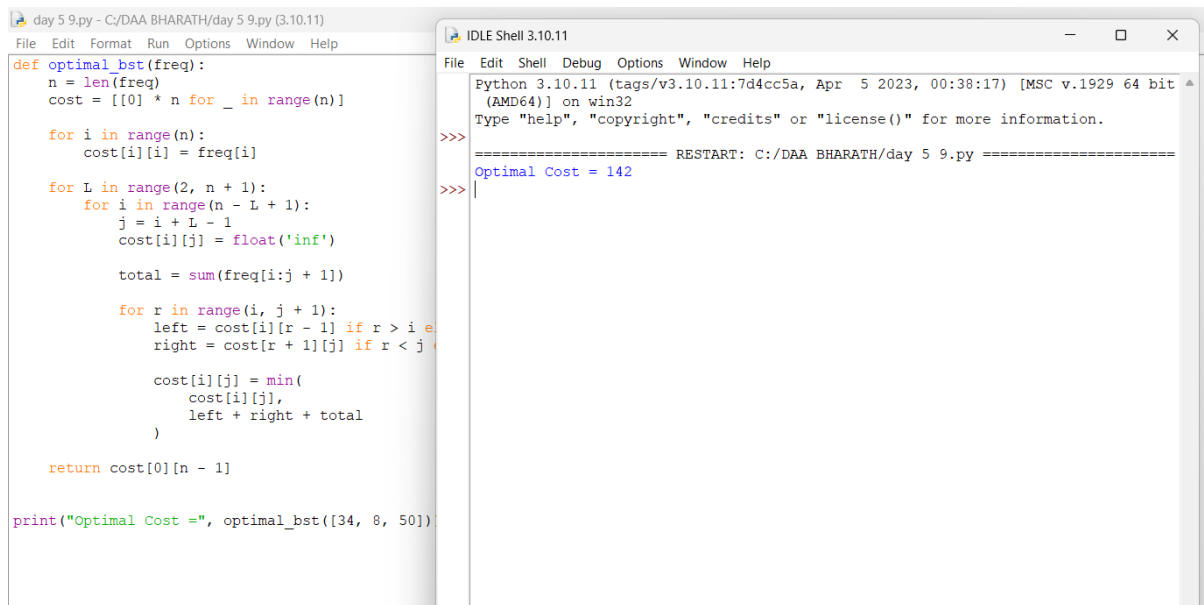
Aim

To implement the Optimal Binary Search Tree using Dynamic Programming and calculate the minimum search cost.

Algorithm

1. Read the frequencies of the keys.
2. Create a cost matrix.
3. Initialize the diagonal values with the corresponding frequencies.
4. Consider subtrees of increasing lengths.
5. Select every key as the root.
6. Calculate left and right subtree costs.
7. Add the total frequency.
8. Store the minimum cost.

Program & Output



```
day 5 9.py - C:/DAA BHARATH/day 5 9.py (3.10.11)
File Edit Format Run Options Window Help

def optimal_bst(freq):
    n = len(freq)
    cost = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for L in range(2, n + 1):
        for i in range(n - L + 1):
            j = i + L - 1
            cost[i][j] = float('inf')

            total = sum(freq[i:j + 1])

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0

                cost[i][j] = min(
                    cost[i][j],
                    left + right + total
                )

            return cost[0][n - 1]

print("Optimal Cost =", optimal_bst([34, 8, 50]))

IDLE Shell 3.10.11
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/DAA BHARATH/day 5 9.py =====
>>> Optimal Cost = 142
>>>
```

10. Matrix Chain Multiplication

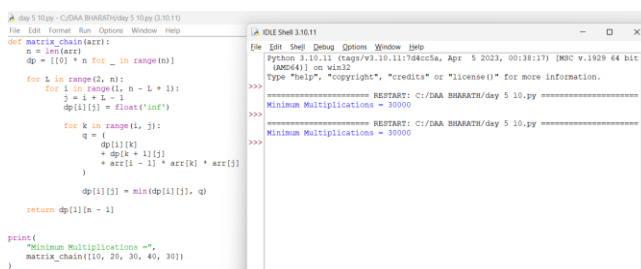
Aim

To implement Matrix Chain Multiplication using Dynamic Programming and find the minimum number of scalar multiplications.

Algorithm

1. Read the dimensions of the matrices.
2. Create a DP table.
3. Consider chains of increasing lengths.
4. Try every possible position for splitting the chain.
5. Calculate the multiplication cost for each split.
6. Store the minimum cost.
7. Return the minimum number of scalar multiplications.

Program & Output



```
day 5 10.py - C:/DAA BHARATH/day 5 10.py (3.10.11)
File Edit Format Run Options Window Help

def matrix_chain(arr):
    n = len(arr)
    dp = [[0] * n for _ in range(n)]

    for L in range(2, n):
        for i in range(1, n - L + 1):
            j = i + L - 1
            dp[i][j] = float('inf')

            for k in range(i, j):
                q = (
                    dp[i][k]
                    + dp[k + 1][j]
                    + arr[i - 1] * arr[k] * arr[j]
                )

                dp[i][j] = min(dp[i][j], q)

            return dp[1][n - 1]

print(
    "Minimum Multiplications =",
    matrix_chain([10, 20, 30, 40, 30])
)

IDLE Shell 3.10.11
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/DAA BHARATH/day 5 10.py =====
>>> Minimum Multiplications = 30000
>>>
===== RESTART: C:/DAA BHARATH/day 5 10.py =====
>>> Minimum Multiplications = 30000
>>>
```

11. Coin Change – Minimum Coins

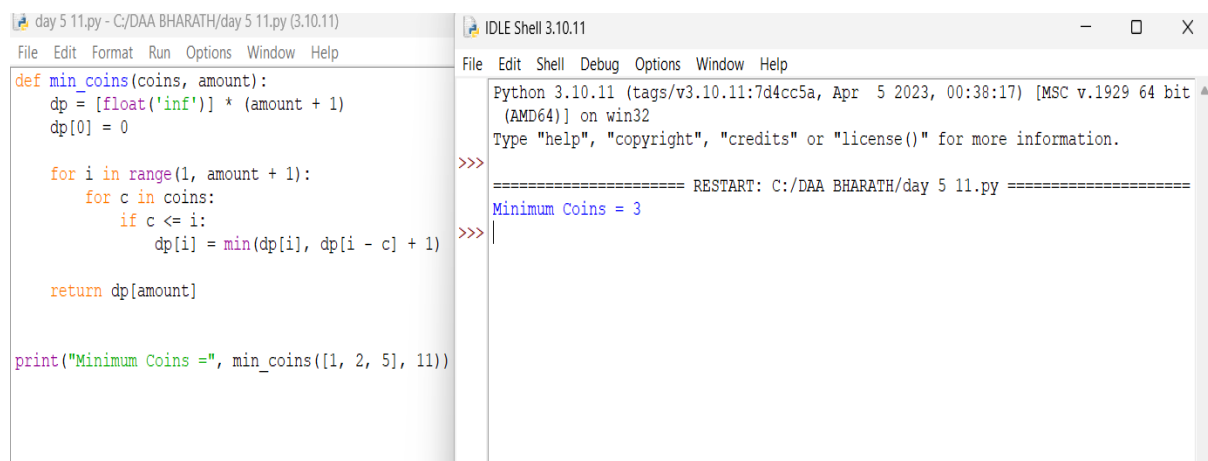
Aim

To implement the Coin Change Problem using Dynamic Programming and find the minimum number of coins required to make a given amount.

Algorithm

1. Read the available coin denominations and amount.
2. Create a DP array.
3. Set $dp[0] = 0$.
4. For every amount, check all available coins.
5. Select the minimum number of coins.
6. Continue until the required amount is reached.
7. Return the minimum number of coins.

Program & Output



```
def min_coins(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i - c] + 1)

    return dp[amount]

print("Minimum Coins =", min_coins([1, 2, 5], 11))
```

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/DAA BHARATH/day 5 11.py =====
Minimum Coins = 3
>>>
```

12. Rod Cutting Problem

Aim

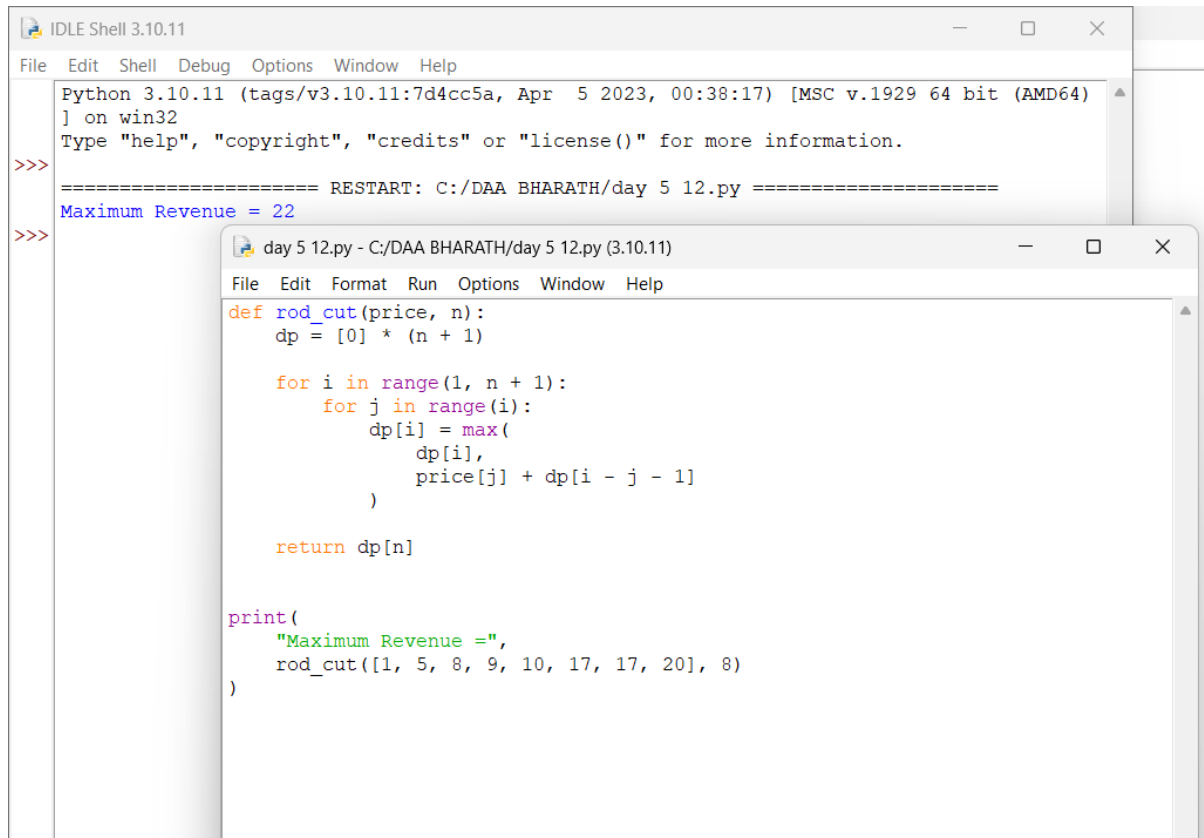
To implement the Rod Cutting Problem using Dynamic Programming and find the maximum revenue obtained by cutting a rod optimally.

Algorithm

1. Read the prices for different rod lengths.
2. Create a DP array.
3. Initialize all values to zero.
4. Consider every possible rod length.
5. Try all possible first cuts.

6. Calculate the maximum revenue for each length.
7. Return the maximum revenue for the complete rod.

Program & Output



The screenshot shows two windows from the Python IDLE 3.10.11 environment. The background window is the 'IDLE Shell 3.10.11', which displays the output of a Python script: 'Maximum Revenue = 22'. The foreground window is the 'day 5 12.py - C:/DAA BHARATH/day 5 12.py (3.10.11)' editor, containing the following Python code:

```
def rod_cut(price, n):
    dp = [0] * (n + 1)

    for i in range(1, n + 1):
        for j in range(i):
            dp[i] = max(
                dp[i],
                price[j] + dp[i - j - 1]
            )

    return dp[n]

print(
    "Maximum Revenue =",
    rod_cut([1, 5, 8, 9, 10, 17, 17, 20], 8)
)
```

13. Minimum Cost Path

Aim

To implement the Minimum Cost Path Problem using Dynamic Programming and find the minimum cost required to reach the bottom-right cell.

Algorithm

1. Read the cost matrix.
2. Create a DP table.
3. Initialize the first cell with its cost.
4. Calculate cumulative costs for the first row.
5. Calculate cumulative costs for the first column.

6. For each remaining cell, add its cost to the minimum of the top and left cells.
7. Return the bottom-right value as the minimum path cost.

Program & Output

```
day 13.py - C:/DAA BHARATH/day 13.py (3.10.11)
File Edit Format Run Options Window Help

def min_cost(cost):
    m = len(cost)
    n = len(cost[0])

    dp = [[0] * n for _ in range(m)]

    dp[0][0] = cost[0][0]

    for i in range(1, m):
        dp[i][0] = dp[i - 1][0] + cost[i][0]

    for j in range(1, n):
        dp[0][j] = dp[0][j - 1] + cost[0][j]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = cost[i][j] + min(
                dp[i - 1][j],
                dp[i][j - 1]
            )

    return dp[m - 1][n - 1]

cost = [
    [1, 3, 1],
    [1, 5, 1],
    [4, 2, 1]
]

print("Minimum Cost =", min_cost(cost))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/day 13.py =====
>>> Minimum Cost = 7
>>> |
```